

CSD430: Server Side Development
Module 11:
Assignment 2: Coding Standards
Isaac Ellingson
8/2/2026

Coding Standards

What are coding standards?

Coding standards are a big umbrella. This can encompass style, patterns, preferences, processes, and even verification requirements.

Style

Style is the most common kind of coding standard. It can be something like the Google Style Guide (<https://google.github.io/styleguide/javaguide.html>). But it could also be something really simple like "we use tabs, not spaces." Style represents the difference you and I have between your hanging / K&R braces and my same-line braces (which coincide with the Google standard and almost all modern Java code). Because different developers have different dialects of a language, different personal preferences about how things are spaced and worded, having "a style guide", or having *organization* coding conventions, or preferably both, can help different developers in the org effectively communicate with each other.

If used wisely, coding style conventions can also generally improve readability and cleanliness. Many of these rules can be checked by automated linters to further take personal preferences and emotions out of the equation and just get to the pure, agreed-upon form of the code with no judgement.

Patterns

This could be, "we prefer dependency injection over client-acquired singletons". Generally a good standard, as the normal singleton pattern can make things hard to unit test in isolation. Every developer has a toolbox of patterns - the visitor pattern, the decorator, flyweight, proxies, adapters / shims, etc. But not every developer can know every pattern or is proficient with it, and your team will find that they are collectively most proficient with a certain set of tools.

So, your team and your codebases will evolve conventions for what patterns are chosen and when these patterns will be used, even if just informally. This will maximize each developer's ability to extend and improve the code.

Preferences

Weird little things wind up happening in a developer shop - everyone uses Windows, or everyone uses Linux, or you've got a 60%/40% split between developers using Eclipse versus bare neovim. Your deployment environment always has a test user with a specific name. I've seen an all-windows shop with like four Macs on it. The mac users had to quickly gain familiarity with how to interoperate with windows machines in order to work effectively with their team. This is a little, overlooked detail, but being clear about these things ahead of time can help you quickly familiarize yourself with a new system (because it's set up "the way we do it"), and help surface potential problems early (mac or intellij user might need extra setup help).

It can also be something **in the code** but not pattern-shaped, like "we always use these standard libraries". If you know you'll have FastUtil, you'll make different design decisions in your code.

Processes

A process standard might be something like, "we get two reviews on a PR before it gets merged", or "on Monday we do pair programming, with the pairs posted for next week on Fridays". Process improvements can make development faster, set limits on how long things can go before being handled or responded to, improve the reliability and security of the product, and improve morale by assigning work more evenly.

Verification Requirements

It can be a standard, for example, that all null pointer access is annotated. Or that all review issues flagged must be addressed by commits before the PR is merged. Maybe code coverage of unit tests needs to be above some threshold. Often these can be automated or linted too.

How Coding Standards Are Enforced

Primarily now we're talking about style manuals and verification requirements. It's outside the scope of a developer class to get into change management or configuration management, but processes exist for these other domains to create process and configuration improvements and make them stick.

Style manuals are typically enforced with linters. One useful example is the checkstyle tool (https://checkstyle.org/google_style.html), which can run automated tests against the Google style manual and ensure your code conforms to it. This can run as part of your project model and CI process, failing the build on the developer's machine before it ever even reaches the repository - and then rechecking it in the repository. The earlier you fail the build for style, the faster the programmer will be able to adapt, and the less code that needs to be changed or rewritten to conform to it.

A less intense version of this is EditorConfig (<https://editorconfig.org/>). Both Eclipse (via a plugin) and IntelliJ iDEA (natively) support this config file. It defines some basic facts about files such as whether trailing whitespace at the ends of lines should be preserved, whether a newline should be inserted at the end of the file to make legacy windows machines happy, and the eternal dispute, whether tabs or spaces should be used and if spaces, *how many*.

Automated linters for material code verification also abound, such as Checker Framework and FindBugs, which I have discussed at length at other times in this course. Just like with CheckStyle, these can be integrated with IDEs and CI to both catch it early and to verify code as it flows through the repository.

Why We Care

Again, this is all about effective communication between programmers. Diving into a new codebase can be intimidating. It'll blast you with a lot of new and unusual information. Setting down some ground rules about what programs should look like, how they should be structured, how they will be orchestrated, what happens during their development cycle, and how they'll be put into production can greatly reduce that mental load and let you find and focus on the actual behaviors you're interested in.

For that reason, on the style front, I highly recommend pointing at a general style guide, and also having a set of "house rules" that are clearly communicated for your org.

References

EditorConfig. (n.d.). In *editorconfig.org*. Retrieved August 2, 2026, from

<https://editorconfig.org/>

GeeksforGeeks. (2019). Coding standards and guidelines. In *GeeksforGeeks*.

<https://www.geeksforgeeks.org/software-engineering/coding-standards-and-guidelines/>

Google. (2019). Google java style guide. In *Github.io*.

<https://google.github.io/styleguide/javaguide.html>

Refactoring Guru. (2014). Design patterns. In *Refactoring.guru*.

<https://refactoring.guru/design-patterns>